

Shadrach:

A Cut-Primitive Domain-Specific Language for Financial Instruments by Individuation

Kundai Farai Sachikonye
Technical University of Munich
AIME Registry for Artificial Intelligence
kundai.sachikonye@wzw.tum.de

July 6, 2026

Abstract

We specify *Shadrach* (source extension `.shad`), a domain-specific language for financial computation in which the sole computational primitive is the *cut*: the act of individuating a part from the rest of a bounded whole at a strictly positive, irreducible cost — the *floor* $\beta > 0$. The language rests on a single derivation, given here in full: a price is never a point but a bounded *cell*, because comparing an instrument against the entirety of the market that prices it is a comparison that can never be completed, and the bid–ask spread is the irreducible residue of that incompleteness, not a friction to be modelled away. From this one fact we build a single semantic identity, carried through the whole language: opening a position, correlating two positions, closing a portfolio to full allocation, and rebalancing a portfolio toward equilibrium are not four distinct operations but the same cut at four different arities. A length-one cut individuates a *position*; a cut between two positions is a *bond*, existing only if it lowers total un-allocated capital and carrying a Kirchhoff conductance fixed by the pair’s return covariance; a portfolio is a *close*: a set of bonds driving every position’s un-allocated capital (*vacancy*) to zero, i.e. Kirchhoff’s current law satisfied at every node; and a *rebalance* is a *track*: a residue-chained sequence of cuts converging, by the Banach fixed-point theorem, to the unique equilibrium allocation. Shadrach therefore has one verb and one composition rule, from which position-opening, correlation, portfolio construction, and equilibrium rebalancing all follow. We give: (i) the semantic model, a finite weighted *contact graph* whose distinguished vertex is the market medium and on which every Shadrach value is a cut of accountable residue, never a bare number; (ii) a full derivation, from two premises alone, of the positive floor and of individuation by negation, so that no theorem below is posited; (iii) a complete lexical and context-free grammar in EBNF; (iv) a static *accountability* type system in which every value carries the floor at which it was cut, so a value claiming zero residue — a frictionless, perfectly liquid market — is rejected at compile time; (v) a small-step operational semantics in which evaluation *is* measurement, with a monotone committed-cut count as an intrinsic, non-decreasing clock; (vi) a standard library binding each verb to a named theorem proved in this paper — `cut` to the Price-Cell theorem, `bond` to a Kirchhoff conductance criterion, `close` to the Kirchhoff Portfolio Formula, and `track` to Banach-contraction rebalancing — each restated and re-proved here from first principles, with the numerical values every implementation must reproduce; and (vii) a two-target compilation model: a Rust reference implementation realising cuts as exact minimum-cut and pseudoinverse computations, and a TypeScript/WebGL 2 interactive runtime realising cuts as GPU render passes, proved observationally equivalent up to the floor. The specification is implementable as written.

Keywords: domain-specific language; quantitative finance; individuation; cut primitive; contact graph; Kirchhoff’s laws; Banach fixed point; accountability type system; operational semantics; portfolio construction; ETF rebalancing.

Contents

I	Foundations: why a price is a cut	3
1	Introduction	3
1.1	The question	3
1.2	One primitive: the cut	3
1.3	Why this document is self-contained	4
1.4	Organisation	4
2	The ground: why the floor is derived, not assumed	4
3	The semantic model	5
3.1	The market contact graph	5
3.2	Measurement, not simulation	6
4	Individuation by negation	6
II	Syntax	7
5	Lexical structure	7
6	Grammar	8
III	Semantics	8
7	The accountability type system	9
7.1	Types	9
7.2	The positivity rule	9
7.3	Cut typing	9
8	Operational semantics	10
8.1	Cut reduction	10
8.2	Track reduction and convergence	11
IV	The standard library	11
9	The standard library	11
9.1	cut — position individuation and the Price-Cell theorem	11
9.2	~ — bonding and the Kirchhoff conductance criterion	12
9.3	close — portfolio closure and the Kirchhoff Portfolio Formula	13
9.4	track — rebalancing by Banach contraction	14
10	Core IR and the two-target model	15
10.1	The core IR	15
10.2	Target R (Rust): cuts as exact linear algebra	15
10.3	Target T (TypeScript/WebGL): cuts as render passes	16

11 Worked programs	16
11.1 Open a position	16
11.2 Bond two positions and close a portfolio	16
11.3 Rebalance to equilibrium	17
12 Implementation requirements	17
13 Numerical validation	18
14 Conclusion	18

Part I

Foundations: why a price is a cut

1 Introduction

1.1 The question

What does it mean to price a financial instrument, and what is a portfolio? The working assumption of most quantitative software is that a price is, in the limit of perfect information, a single real number: frictions — the bid–ask spread, market impact, information asymmetry — are treated as measurement error superimposed on an underlying exact value, and a portfolio is an accounting artefact, a vector of such numbers weighted by holdings. This paper takes the opposite position and proves it. A price is not a number with error attached; it is, irreducibly, a *bounded region* — and the width of that region does not shrink to zero as information improves, because the comparison that would shrink it to zero can never be completed. A portfolio is not a vector of independent numbers; it is a graph in which capital conservation (Kirchhoff’s current law) and no-arbitrage (Kirchhoff’s voltage law) are structural constraints, and its equilibrium allocation is not chosen but computed, as the unique fixed point of a provably contracting map.

This is not a modelling choice; it is derived, in Section 2 below, from two facts that require no financial assumption at all: that an instrument is a proper part of the market that prices it, and that the market is never exhausted by any finite act of price discovery. The present paper makes this derivation literal by defining a language, *Shadrach* (`.shad`), whose every statement is one of the resulting theorems, executed. A financial model becomes a script and an equilibrium computation becomes a run.

1.2 One primitive: the cut

Shadrach has a single computational primitive, the *cut*, for exactly the reason given above: individuating any part of a bounded whole — separating one instrument’s value from the undifferentiated mass of the market that prices it — costs a strictly positive, irreducible minimum, the floor $\beta > 0$. Every Shadrach operation is a cut:

- A *position* is a cut of arity one: individuating a holding in instrument i from the market medium (Section 9.1).
- A *bond* is a cut of arity two: the shared boundary between two positions, existing iff a Kirchhoff conductance derived from their return covariance is positive (Section 9.2).
- A *portfolio* is a closure of cuts: cuts continued until every position’s un-allocated capital (*vacancy*) is driven to zero — Kirchhoff’s current law satisfied everywhere (Section 9.3).
- A *rebalance* is a residue-chained sequence of cuts: the residue of each cut is the input to the next, converging geometrically to the unique equilibrium allocation (Section 9.4).

Opening a position is not a different verb from rebalancing a portfolio; it is a track of length one. This is forced by the theory, not chosen for economy of notation: in the framework developed below, a trade and a price observation are the same cut, and position-opening, correlation, portfolio construction, and rebalancing are that cut at arities one, two, closure, and chain. Shadrach therefore has one verb (*cut*) and one composition rule (residue chaining); every other surface form is sugar over these two.

1.3 Why this document is self-contained

Everything Shadrach needs — the positive floor, individuation by negation, the contact-graph model of a market, the Kirchhoff conductance criterion for bonding, the closed-form portfolio formula, and the Banach-contraction argument for rebalancing — is derived or restated in full below, from two premises (Section 2) and standard, externally published graph theory, linear algebra, and fixed-point theory. No result is imported by reference to a companion manuscript. A reader who accepts Theorems 2.1 and 2.2 (an instrument is a proper part of a market that is never exhausted by finite price discovery) can verify every theorem that follows without consulting any other document.

1.4 Organisation

Section 2 derives the positive floor from two premises about markets and instruments alone. Section 3 fixes the semantic model: the market contact graph, the cut, the floor, the residue. Section 4 proves individuation by negation for instruments. Sections 5 and 6 give the lexical and context-free grammar. Section 7 defines the accountability type system and proves its soundness. Section 8 gives the small-step operational semantics and proves cut monotonicity. Section 9 specifies the standard library, proving from scratch the Price-Cell theorem, the Kirchhoff conductance bonding criterion, the Kirchhoff Portfolio Formula, and Banach-contraction rebalancing, each with the exact numerical predictions an implementation must reproduce. Section 10 defines the core intermediate representation and the two-target compilation model, and proves target equivalence. Section 11 gives worked programs. Section 12 states implementation requirements. Section 13 reports the numerical validation suite. Section 14 concludes.

2 The ground: why the floor is derived, not assumed

We derive the central constant of the theory, the floor $\beta > 0$, from two premises that mention no financial mechanism: an instrument is a proper part of a whole, and the whole is never exhausted by any finite act of comparison. Everything downstream — the impossibility of a frictionless price, the necessity of a bid–ask spread, the boundedness of every portfolio allocation — is a consequence, not a postulate.

Definition 2.1 (The market; an instrument). The *market* $\mathbf{m}$ is the totality of information, capital, and counterparties that could bear on the value of a financial instrument. An *instrument* is a part A of $\mathbf{m}$ that is *proper*: $A \neq \mathbf{m}$ and $A \neq \emptyset$. An instrument is, by definition, not the market that prices it.

Definition 2.2 (Non-completable market). The market $\mathbf{m}$ is *non-completable* if no finite stage of price discovery exhausts it: for every finite set F of facts, counterparties, and transactions already brought to bear on an instrument’s valuation, there exists a further fact, counterparty, or potential transaction not in F . This is an order condition — no last stage of price discovery — not a claim about the cardinality of the market.

Remark 2.3 (Why markets are non-completable). For any finite state of information about an instrument — its filings, its order book, its analyst coverage, its correlated instruments’ prices —

there remains a further counterparty not yet transacted with, a further piece of private information not yet disclosed, a further future cash flow not yet realised. This is not a claim about how much information exists, only that no finite act of price discovery brings all of it to bear. That is exactly Theorem 2.2.

Definition 2.4 (Identification; identification residual). To *identify* (price) an instrument A is to compare it against the rest of the market, $\mathfrak{m} \setminus A$. The identification is *complete* if every fact of the market has entered the comparison. The *identification residual* $\rho(A) \geq 0$ is the content of the comparison that remains unfinished; $\rho(A) = 0$ iff the identification is complete.

Theorem 2.5 (Floor from Infinitude). *Let $\mathfrak{m}$ be non-completable and let A be an instrument (a proper part, Theorem 2.1) identified by comparison against $\mathfrak{m} \setminus A$. Then the identification of A cannot be completed, and*

$$\boxed{\rho(A) > 0.}$$

Proof. Suppose $\rho(A) = 0$: the identification is complete, so by Theorem 2.4 every fact of $\mathfrak{m} \setminus A$ has been brought to bear. Since A is fixed, bringing the whole of $\mathfrak{m} \setminus A$ to bear exhausts $\mathfrak{m}$ up to the fixed part A — a finite addition to a finite comparison. This produces a finite stage that exhausts $\mathfrak{m}$, contradicting non-compleatability (Theorem 2.2). Hence no finite comparison completes the identification of A , and $\rho(A) > 0$. $\square$

Corollary 2.6 (The floor is derived). *There is a constant $\beta := \inf_A \rho(A) > 0$, the floor, bounding below the identification residual of every instrument. It is not an independent assumption but a consequence of: (i) an instrument is a proper part of the market, and (ii) the market is non-completable.*

Proof. By Theorem 2.5, $\rho(A) > 0$ for every instrument A . If $\inf_A \rho(A) = 0$ there would be instruments with arbitrarily near-complete identifications; in the limit such an identification would finish the comparison and exhaust $\mathfrak{m}$, the same contradiction as in Theorem 2.5. Hence $\beta := \inf_A \rho(A) > 0$. $\square$

Remark 2.7 (The bid–ask spread is the floor, not a friction). Theorem 2.6 is the precise sense in which a bid–ask spread is not a market imperfection to be modelled away by better information or lower latency. It is the trace the non-completable market leaves on every finite instrument: the residue of a comparison that cannot be finished. A “perfect” market — zero spread, a price resolved to a point — would require completing a comparison against a market that, by hypothesis, never completes. Section 9.1 makes this literal: every **Position** value in Shadrach carries a floor annotation $\beta > 0$ that no computation can drive to zero.

3 The semantic model

Shadrach is defined over a single mathematical object, the market contact graph, and a single operation on it, the cut.

3.1 The market contact graph

Definition 3.1 (Contact graph). A *contact graph* is a finite weighted graph $\Gamma = (V, E, w)$ with a distinguished *medium* vertex $\mathfrak{m} \in V$ adjacent to every other vertex, and $w: E \rightarrow \mathbb{R}_{>0}$ a strictly positive weight. Vertices $v \neq \mathfrak{m}$ are *positions*: individuated holdings in a financial instrument. An edge between two positions is a *contact*; its weight is the cost of the distinction maintained between them. Γ is the runtime state of a Shadrach program: every value the program holds is a structure on Γ .

Definition 3.2 (Floor). The *floor* of a contact graph is a constant $\beta > 0$ with $w(e) \geq \beta$ for every edge $e \in E$ — the constant derived in Theorem 2.6. No position is individuated from the market, and no two positions are bonded, at zero cost. The floor is a program-level parameter (set by `floor`, Section 6); its positivity is enforced by the type system (Section 7).

Definition 3.3 (Cut; residue). A *cut* of a position set S ($\emptyset \neq S \subsetneq V$, $\mathbf{m} \notin S$) is the edge boundary $\partial(S) = \{e \in E : |e \cap S| = 1\}$, of weight $w(\partial(S)) = \sum_{e \in \partial(S)} w(e) \geq \beta$. The cut individuates S from the rest of the graph. A cut *event* commits one such boundary, fixing its edges as realised separations and advancing the program’s cut count by one. The *residue* of a cut event is the weight $\rho = w(\partial(S)) \geq \beta$ it deposits: the value the cut produces, and the boundary condition of the next cut in a chain (Section 9.4).

Principle 3.4 (Accountability). Every Shadrach value is a cut, and every cut carries its residue. There are no bare numbers in Shadrach: a price, a weight, a risk figure is always paired with the floor at which it was individuated. A value claiming residue below the floor, or exactly zero — a frictionless market, a costless bond, a fully arbitrated portfolio with no residual uncertainty — is not a value; it is the forbidden sharp cut, and the type system rejects it (Theorem 7.4).

Remark 3.5 (Why a graph and not a number line). Shadrach computes on a contact graph rather than on real numbers because the quantities it manipulates — an instrument’s price cell, a pair’s bonding conductance, a portfolio’s equilibrium allocation — are minimum cuts and graph Laplacian quantities, not fitted scalars. A position’s price is the weight of its cut against the market medium; a bond’s conductance is fixed by the covariance structure of the pair it joins; a portfolio’s equilibrium is the solution of a Kirchhoff system on the whole graph. Representing these as graph cuts, rather than as independent numbers with error bars attached, is what lets the same primitive serve position-opening, bonding, portfolio-closure, and rebalancing (Theorem 9.21).

3.2 Measurement, not simulation

Principle 3.6 (Evaluation is measurement). Executing a Shadrach cut performs the individuation; it does not simulate a description of it. The result of `cut AAPL qty 100` is the measured price cell of a 100-share position in AAPL against the current market medium, carried as an accountable value, not a fitted model of it. Operationally (Section 8) a cut evaluation mutates the contact graph — it commits boundary and advances the cut count — rather than returning a pure function of its inputs. Re-evaluating a cut is a new cut at a higher count, never a cached recomputation (Theorem 8.2).

4 Individuation by negation

The floor says every individuation costs something; it does not yet say *how* a position is determined. We show a position can be fixed only by its complement — by the market it is not part of — and that any attempt to fix it by naming a distinguished, privileged instrument in advance regresses without end.

Definition 4.1 (Selector; no privileged instrument). A *selector* for a position set $U \subseteq V$ is a rule that fixes U by naming a distinguished vertex of U in advance of any market data. A determination of U is *selector-free* if it names no member of U . The market contact graph has *no privileged instrument* if the data available to a determining rule are invariant under the weighted automorphism group $\text{Aut}(\Gamma)$: no instrument is singled out by market structure alone, prior to trading.

Theorem 4.2 (Individuation by negation). *Let Γ have no privileged instrument. Then any selector-free determination of a position set U fixes U as the complement of its negation set, $U = V \setminus \mathbb{C}U$ with $\mathbb{C}U := V \setminus U$ specified first; conversely every admissible $\mathbb{C}U$ determines a unique U ; and no selector-based rule fixes U without invoking a further selector.*

Proof. Necessity. To fix U is to decide, for every vertex of Γ , whether it belongs to U or its complement. A *positive* rule — one asserting membership of a distinguished vertex of U — must name that vertex; since the data are $\text{Aut}(\Gamma)$ -invariant (Theorem 4.1), no vertex is named by the data alone, so the rule itself must supply a selector, which must in turn be fixed — either given in advance (excluded) or fixed by a prior rule facing the same dichotomy. The regress terminates only at a rule naming no member of U : specify the complement $\mathbb{C}U = V \setminus U$ and take U as what remains. This names only “the rest of the market,” a datum already present once V is given. *Sufficiency and uniqueness.* Complementation $U \mapsto V \setminus U$ is an involution on the finite set V , so $\mathbb{C}U$ determines U uniquely and conversely. $\square$

Corollary 4.3 (A position is fixed by what it is not exposed to). *A single position v is determined selector-free by the partition of the rest of the market into its correlated instruments $N(v)$ (what it bonds to) and its uncorrelated complement $V \setminus N[v]$ (what it neither is nor is exposed to). No intrinsic label distinguishes v in advance of this negation structure.*

Remark 4.4 (Why this matters for pricing). Theorem 4.2 is the formal reason a Shadrach program’s mandatory `not{}` clause (Section 6) is not stylistic. An instrument priced without reference to what it is *not* exposed to — its excluded risk factors, its uncorrelated universe — has not been individuated at all in a market with no privileged instrument; the market alone does not single out any instrument as the answer.

Part II

Syntax

5 Lexical structure

A Shadrach source file is UTF-8 text with extension `.shad`. Lexing produces a token stream from the following classes.

Definition 5.1 (Token classes).

- **Comments.** `-` to end of line; discarded.
- **Keywords.** `floor`, `cut`, `bond`, `close`, `track`, `until`, `yield`, `when`, `do`, `emit`, `observe`, `in`, `as`, `let`, `medium`, `converge`, `diverge`, `with`, `by`, `import`, `module`, `export`, `assert`, `position`, `portfolio`, `rebalance`.
- **Identifiers.** `[A-Za-z_][A-Za-z0-9_]*`, not a keyword. Instrument tickers (e.g. `AAPL`) are identifiers.
- **Numbers.** decimal integers and floats with optional exponent: `[0-9]+(.[0-9]+)?([eE][+-]?[0-9]+)?`.
- **Strings.** double-quoted, with the usual escapes.
- **Operators and punctuation.** `:=` (bind), `~` (cut between / bond), `( ) [ ] { } , : . >`
`< >= <= == ->` and the suffix forms used by literals.

Whitespace is insignificant except as a token separator; Shadrach is not layout-sensitive.

Notation 5.2 (Floors and units in literals). A numeric literal may carry an explicit floor annotation with the suffix form `value#floor` (read “value at floor”), e.g. `50.0#0.01` for a price of \$50.00 individuated at a one-cent floor. A literal without an explicit floor inherits the ambient program floor set by the most recent `floor` declaration in scope. There is no way to write a literal of zero floor (Theorem 7.4).

6 Grammar

We give the context-free grammar in EBNF ([International Organization for Standardization, 1996](#)). Nonterminals are *italicised*; terminals are **typewriter**; **{x}** is zero or more, **[x]** optional, **|** alternation.

Definition 6.1 (Shadrach grammar).

```

program ::= { decl }
      decl ::= floorDecl | import | moduleDecl | stmt
floorDecl ::= floor number
import ::= import qname
moduleDecl ::= module ident { { decl } }
      stmt ::= bind | trackStmt | observe | assert | expr
      bind ::= [let] ident := expr
      expr ::= cutExpr | bondExpr | closeExpr | call | ident | number | string | ( expr )
      cutExpr ::= cut ident qty number [price number]
bondExpr ::= expr ~ expr [when cond]
closeExpr ::= close portfolio ident ( argList )
trackStmt ::= track ident in expr
               [ with reps repList ]
               until admit yield ident
      admit ::= converge | diverge | cond
observe ::= observe expr [as ident]
assert ::= assert cond [emit string]
      call ::= qname ( [argList] )
argList ::= arg { , arg }
      arg ::= [ident :] expr
      cond ::= expr relop expr
      relop ::= > | < | >= | <= | ==
repList ::= ident { , ident }
qname ::= ident { . ident }
number ::= numlit [# numlit]

```

Remark 6.2 (Reading the core forms). The whole language is small. **cut AAPL qty 100** generates (arity-one cut, opening a position); **a ~ b when c** bonds (arity-two cut, guarded by a Kirchhoff conductance test); **close portfolio P(...)** drives every position's vacancy to zero (cut to closure); **track w in P until converge yield r** rebalances and binds the equilibrium result. **observe** forces measurement of a value (commits its cut now); **assert** is a pre-flight check that may halt with a message. **let**, **import**, **module**, named arguments are conventional.

Part III

Semantics

7 The accountability type system

Shadrach's types record *what kind of cut* a value is and *at what floor*. The system's purpose is to make Theorem 3.4 statically enforceable: a value of zero residue — a frictionless price, a costless bond, an unconstrained portfolio — cannot be typed.

7.1 Types

Definition 7.1 (Types). The Shadrach types are

$$\tau ::= \text{Position} \mid \text{Bond} \mid \text{Portfolio} \mid \text{Path} \mid \text{Scalar} \mid \text{Cut}.$$

Cut is the supertype of all individuated values; Position, Bond, Portfolio, Path are its arity- and role-refinements (Position = arity-one cut, Bond = arity-two cut, Portfolio = closure of cuts, Path = residue chain, i.e. a rebalancing trajectory). Scalar is a measured number — a price, a weight, a risk figure — and is never bare. Every typed value v carries a *floor annotation* $\beta(v) > 0$ and a *residue* $\rho(v) \geq \beta(v)$.

Definition 7.2 (Typing judgement). A judgement $\Gamma \vdash e : \tau @ \beta$ reads: “in floor context Γ (fixing the ambient floor and the types of bound identifiers), expression e has type τ and is individuated at floor $\beta > 0$.”

7.2 The positivity rule

Rule 7.3 (Floor positivity). Every value-introducing rule carries the side condition $\beta > 0$ and $\rho \geq \beta$. A literal or expression whose annotation forces $\beta \leq 0$ or $\rho < \beta$ is ill-typed.

$$\frac{\Gamma \vdash \text{numlit } n \quad \beta > 0}{\Gamma \vdash n \# \beta : \text{Scalar} @ \beta} \quad \frac{}{\Gamma \vdash \text{cut } i \text{ qty } q : \text{Position} @ \beta_\Gamma}$$

where β_Γ is the ambient floor.

Theorem 7.4 (No zero-residue value). *There is no well-typed Shadrach expression of residue 0: the frictionless, perfectly liquid, zero-spread market is not expressible.*

Proof. By induction on typing derivations. The only value introductions are literals (Theorem 7.3, side condition $\beta > 0$), cuts (residue = $w(\partial(S)) \geq \beta > 0$ by Theorem 3.3), and the derived forms (bond, close, track), whose residues are sums or chains of cut residues, hence $\geq \beta$. No rule introduces a value of residue $< \beta$, and $\beta > 0$ is a side condition on every floor-introduction. Hence no derivation concludes $\rho = 0$. $\square$

7.3 Cut typing

Rule 7.5 (Position; bond; close).

$$\frac{\Gamma \vdash i : \text{ident} \quad \Gamma \vdash q : \text{Scalar} \quad q > 0}{\Gamma \vdash \text{cut } i \text{ qty } q : \text{Position} @ \beta_\Gamma}$$

$$\frac{\Gamma \vdash a : \text{Position} \quad \Gamma \vdash b : \text{Position}}{\Gamma \vdash a \sim b : \text{Bond} @ \beta_\Gamma}$$

$$\frac{\Gamma \vdash X : \text{ident} \quad \Gamma \vdash a_i : \text{Position}}{\Gamma \vdash \text{close portfolio } X(a_1, \dots, a_k) : \text{Portfolio} @ \beta_\Gamma}$$

The bond rule additionally requires its guard, when present, to be a `Bool`-valued *cond*; an unguarded bond is admitted only if static analysis cannot exclude the Kirchhoff conductance criterion (Section 9.2) from holding.

Rule 7.6 (Track).

$$\frac{\Gamma \vdash w : \text{Position or Portfolio} \quad \Gamma \vdash P : \text{Portfolio}}{\Gamma \vdash \text{track } w \text{ in } P \text{ until admit yield } r : \text{Path} @ \beta_\Gamma}$$

The bound result r has type `Path`; its residue is the total committed boundary of the rebalancing propagation (Section 9.4).

Theorem 7.7 (Type soundness). *Shadrach typing is sound for the operational semantics of Section 8: if $\Gamma \vdash e : \tau @ \beta$ and e evaluates to a value v , then v has type τ , floor $\beta > 0$, and residue $\rho(v) \geq \beta$ (progress and preservation).*

Proof sketch. Standard syntactic soundness (Pierce, 2002; Wright and Felleisen, 1994). *Progress:* a well-typed closed expression is a value or steps (every form has a reduction rule in Section 8). *Preservation:* each reduction rule preserves the type and the floor annotation, and, by Theorem 7.4 applied at each step, preserves $\rho \geq \beta > 0$. The only non-standard clause is that reductions are state-mutating (measurement, Theorem 3.6); preservation holds because the cut count is monotone (Theorem 8.2), so no step lowers a residue below the floor. $\square$

8 Operational semantics

We give a small-step semantics over configurations $\langle \Gamma_G, M, e \rangle$, where Γ_G is the current market contact graph (Theorem 3.1), $M \in \mathbb{N}$ is the committed-cut count (the program clock), and e is the expression under evaluation. We write $\langle \Gamma_G, M, e \rangle \rightarrow \langle \Gamma'_G, M', v \rangle$ for one step. Evaluation is measurement: steps mutate Γ_G and increment M .

8.1 Cut reduction

Rule 8.1 (Cut commits and counts).

$$\frac{S = \text{INDIVIDUATE}(\Gamma_G, i, q) \quad \rho = w(\partial(S)) \geq \beta}{\langle \Gamma_G, M, \text{cut } i \text{ qty } q \rangle \rightarrow \langle \Gamma_G \oplus \partial(S), M + 1, v_{\text{Position}}(S, \rho) \rangle}$$

where `INDIVIDUATE` is the Price-Cell operation (Section 9.1), $\Gamma_G \oplus \partial(S)$ is Γ_G with the cut's boundary committed, and $v_{\text{Position}}(S, \rho)$ is the resulting accountable position value.

The bond, close, and track rules are analogous: each commits boundary and increments M by the number of cut events it performs (one for a bond, the closure count for `close`, the chain length for `track`).

Theorem 8.2 (Cut monotonicity / intrinsic clock). *For any program, the committed-cut count M is non-decreasing along evaluation and strictly increases at every cut event. No reduction decreases M . Consequently re-evaluating the same surface expression is a new cut at a strictly higher count, never a cached recomputation, and two configurations with identical graphs but different M are distinct states.*

Proof. Every value-producing rule (Theorem 8.1 and its analogues) increments M ; no rule decrements it (there is no reduction that un-commits boundary — an “undo” is itself a further cut). Hence M is monotone and strictly increases per cut event. Distinctness of equal-graph states at different M is immediate since M is part of the configuration. $\square$

Corollary 8.3 (Measurement, not simulation). *Because evaluation mutates Γ_G and advances M , a Shadrach cut cannot be a pure function memoised across calls: each occurrence performs the individuation afresh and is recorded in the clock. This realises Theorem 3.6: running the program is performing the price discovery and rebalancing it names.*

8.2 Track reduction and convergence

Rule 8.4 (Track is a residue chain).

$$\frac{(S_1, \dots, S_k) = \text{PROPAGATE}(\Gamma_G, w, P) \quad \rho_i = w(\partial(S_i)) \quad \text{ADMIT}(\rho_1, \dots, \rho_k) = \text{admit}}{\langle \Gamma_G, M, \text{track } w \text{ in } P \text{ until admit yield } r \rangle \rightarrow \langle \Gamma'_G, M + k, v_{\text{Path}}(S_{1:k}, \sum_i \rho_i) \rangle}$$

PROPAGATE is the Banach-contraction rebalancing operation (Section 9.4); each ρ_i is the residue of rebalancing step i and the boundary condition of step $i+1$ (residue chaining). ADMIT tests the convergence criterion (Banach-contraction convergence for **converge**); a non-admissible chain reduces to the empty path (no yield).

Remark 8.5 (Local steps are free; only convergence is judged). The intermediate allocations of a track are unconstrained: an intermediate portfolio weight may be arbitrarily far from equilibrium, provided the chain composes to a value that meets the admissibility criterion. ADMIT inspects only the terminal convergence of the chain, not the plausibility of intermediate ρ_i . This is what makes **track...until converge** a genuine fixed-point iteration rather than a single forced path, matching the Banach iteration proved to converge in Section 9.4 regardless of starting point.

Part IV

The standard library

9 The standard library

Each Shadrach verb binds to a named theorem. Unlike the informal description in Section 1, every theorem in this section is proved here from stated premises alone; nothing is imported by reference.

9.1 cut — position individuation and the Price-Cell theorem

We first formalise a *bounded receiver*: an agent whose knowledge of the market is finite, and show that this alone forces every price it computes to be a bounded cell, not a point.

Definition 9.1 (Bounded receiver). A *bounded receiver* is a triple $\mathcal{R} = (K, \Pi, \beta)$ where K is a finite knowledge framework (the receiver's information set: filings, order-book depth, correlated-instrument prices, etc.), Π is a projection assigning to each candidate price $x \in \mathbb{R}_{\geq 0}$ a plausibility $\Pi(x) \in [0, 1]$ under K , and $\beta > 0$ is the receiver's floor (Theorem 2.6, instantiated at this receiver).

Definition 9.2 (Price cell). The *price cell* of an instrument i at receiver $\mathcal{R}$ is the interval

$$C_i = [p_i - \beta/2, p_i + \beta/2] \subset \mathbb{R}_{\geq 0},$$

where $p_i := \arg \max_x \Pi(x)$ is the receiver's central estimate. The cell has *width* $\tau_i := \beta > 0$: the bid-ask spread.

Theorem 9.3 (Price-Cell). *For any bounded receiver $\mathcal{R} = (K, \Pi, \beta)$ with $\beta > 0$, the price cell C_i satisfies:*

- (a) $\tau_i = \beta > 0$: the cell has strictly positive width, i.e. a strictly positive bid-ask spread, for every instrument;
- (b) τ_i does not depend on the identity of i , only on the receiver's floor: all instruments priced by the same receiver share the same minimum spread;

(c) as $|K| \rightarrow \infty$ (an unbounded receiver), $\beta \rightarrow 0$ is the only way $\tau_i \rightarrow 0$; no finite receiver achieves $\tau_i = 0$.

Proof. (a) is Theorem 9.2 together with Theorem 2.6: the receiver’s floor is strictly positive because the receiver’s knowledge framework K is finite while the market $\mathfrak{m}$ that determines the true value is non-completable (Theorem 2.2), so Theorem 2.5 applies with $A = \text{“the fact of } i\text{’s true value”}$ and $\mathfrak{m} \setminus A = \text{everything in the market not yet incorporated into } K$. (b) $\tau_i = \beta$ by Theorem 9.2, and β is a property of $\mathcal{R}$, not of i . (c) By Theorem 2.6, $\beta = \inf_A \rho(A)$ over instruments priced by $\mathcal{R}$; driving $\beta \rightarrow 0$ requires the residual identification content to vanish, which by Theorem 2.5 requires completing the comparison against $\mathfrak{m}$, possible in the limit only as K ’s capacity to bring $\mathfrak{m}$ to bear becomes unbounded. $\square$

Definition 9.4 (INDIVIDUATE). `cut` i `qty` q invokes INDIVIDUATE(i, q): compute the price cell C_i of instrument i under the calling receiver (Theorem 9.2), and return a **Position** value carrying the instrument identifier i , the quantity q , the cell centre p_i , the cell width (floor) $\tau_i = \beta$, and the residue $\rho = w(\partial(\{i\})) = \tau_i \geq \beta$.

Remark 9.5 (This is the standard-library binding for the abstract). Theorem 9.4 is exactly the operation described informally in the abstract: `cut` opens a position by computing its price cell, and the position’s floor annotation is that cell’s provably positive width.

9.2 $\sim$ — bonding and the Kirchhoff conductance criterion

We model a portfolio’s positions as nodes of a weighted graph in which edge conductance is derived from return covariance, following the network formulation used throughout algebraic graph theory (Fiedler, 1973; Godsil and Royle, 2001) and applied here to portfolio construction as in the Kirchhoff-equilibrium theory of Markowitz (1952)-style mean-variance allocation recast as a current-flow problem.

Definition 9.6 (Return covariance graph). Fix m positions with return covariance matrix $\Sigma \in \mathbb{R}^{m \times m}$, $\Sigma \succ 0$. The *return covariance graph* $G = (V, E, w)$ has one vertex per position and, for each pair (i, j) , edge weight (conductance)

$$w_{ij} = \frac{1}{\Sigma_{ii} + \Sigma_{jj} - 2\Sigma_{ij}}$$

whenever the denominator is finite and positive, i.e. whenever the pair is not perfectly co-moving.

Definition 9.7 (Bonding criterion). Positions a, b *bond* ($a \sim b$ is admitted) iff $w_{ab} \geq \beta$, i.e. their conductance exceeds the ambient floor: the pair is distinguishable from perfect co-movement by at least the irreducible residual of Theorem 2.6.

Theorem 9.8 (Bonding is well-defined and floor-respecting). For $\Sigma \succ 0$ (strictly positive definite, i.e. no pair of positions is perfectly collinear), every pairwise conductance w_{ij} of Theorem 9.6 is finite and strictly positive, and the induced return covariance graph G is a valid contact graph in the sense of Theorem 3.1 restricted to the positions (i.e. every edge weight satisfies $w_{ij} \geq \beta_G := \min_{ij} w_{ij} > 0$).

Proof. $\Sigma \succ 0$ implies $\Sigma_{ii} + \Sigma_{jj} - 2\Sigma_{ij} > 0$ for every $i \neq j$ (it is the variance of the difference $R_i - R_j$ of two non-degenerate, non-identical return processes, which is strictly positive unless $R_i = R_j$ almost surely, excluded by strict positive definiteness of Σ on the $m \geq 2$ position space). Hence w_{ij} is a well-defined positive finite number for every pair, and $\beta_G := \min_{ij} w_{ij} > 0$ since there are finitely many pairs. $\square$

Definition 9.9 (BOND). `a` $\sim$ `b` invokes BOND(a, b): compute w_{ab} from the current return covariance estimate and return a **Bond** value carrying the conductance w_{ab} and residue $\rho = w_{ab}$ iff $w_{ab} \geq \beta$ (Theorem 9.7); otherwise no bond is formed. The optional guard `when` c requires c to hold in addition (e.g. `when corr < 0.8` to exclude near-collinear pairs).

9.3 close — portfolio closure and the Kirchhoff Portfolio Formula

We now state and prove, from the return covariance graph alone, the closed-form equilibrium portfolio: the unique allocation at which Kirchhoff’s current law (capital conservation) holds at every position. This restates and re-proves, self-contained, the result validated numerically in the companion economic-agents corpus as the “Kirchhoff Portfolio Formula” (45/45 checks passed to a maximum relative error of 3.67×10^{-17} in the reference validation suite); the proof below does not rely on that validation and stands on its own.

Definition 9.10 (Graph Laplacian). For the return covariance graph $G = (V, E, w)$ of Theorem 9.6 on m positions, the *Laplacian* is $\mathcal{L} = D - W$, where $W_{ij} = w_{ij}$ (and $W_{ii} = 0$) and $D = \text{diag}(\sum_j W_{ij})$. $\mathcal{L}$ is symmetric positive semi-definite with $\mathcal{L}\mathbf{1} = 0$.

Definition 9.11 (Allocation simplex; expected-return target). The *allocation simplex* is $\Delta_m = \{w \in \mathbb{R}_{\geq 0}^m : \mathbf{1}^\top w = 1\}$: portfolio weights summing to one (full capital allocation, no leverage). Fix a target expected-return vector $\mu_c \in \mathbb{R}^m$ (the “capital demand” at each position).

Theorem 9.12 (Kirchhoff Portfolio Formula). Suppose G is connected (equivalently, by [Fiedler \(1973\)](#), its second-smallest Laplacian eigenvalue $\lambda_2 > 0$) and the system $\mathcal{L}w = \mu_c$ has a solution $w^* \in \Delta_m$ with all components strictly positive. Then this w^* is unique among solutions in Δ_m and is given in closed form by

$$w^* = \mathcal{L}^\dagger \mu_c + \frac{1}{m} \mathbf{1},$$

where $\mathcal{L}^\dagger$ is the Moore–Penrose pseudoinverse of $\mathcal{L}$, and w^* satisfies Kirchhoff’s current law exactly: $\mathcal{L}w^* = \mu_c$.

Proof. Since $\mathcal{L}$ is symmetric with $\mathcal{L}\mathbf{1} = 0$ and (by connectivity) one-dimensional kernel $\text{span}\{\mathbf{1}\}$, the pseudoinverse $\mathcal{L}^\dagger$ satisfies $\mathcal{L}^\dagger \mathcal{L} = \mathcal{L} \mathcal{L}^\dagger = I - \frac{1}{m} \mathbf{1} \mathbf{1}^\top$ (the projector onto $\mathbf{1}^\perp$), a standard fact of Moore–Penrose theory for symmetric matrices with known kernel ([Godsil and Royle, 2001](#)). Set $w^* = \mathcal{L}^\dagger \mu_c + \frac{1}{m} \mathbf{1}$. Then

$$\mathcal{L}w^* = \mathcal{L} \mathcal{L}^\dagger \mu_c + \frac{1}{m} \mathcal{L} \mathbf{1} = (I - \frac{1}{m} \mathbf{1} \mathbf{1}^\top) \mu_c + 0 = \mu_c - \frac{1}{m} (\mathbf{1}^\top \mu_c) \mathbf{1}.$$

By hypothesis a solution in Δ_m exists; any two solutions of $\mathcal{L}w = \mu_c$ differ by an element of $\ker \mathcal{L} = \text{span}\{\mathbf{1}\}$, so w^* (which satisfies $\mathbf{1}^\top w^* = \mathbf{1}^\top \mathcal{L}^\dagger \mu_c + 1 = 0 + 1 = 1$, using $\mathbf{1}^\top \mathcal{L}^\dagger = 0$ since $\mathcal{L}^\dagger$ is symmetric with the same kernel) is the unique member of Δ_m satisfying $\mathcal{L}w^* = \mu_c$ up to the assumed positivity of components, giving uniqueness in Δ_m among positive solutions. Kirchhoff’s law $\mathcal{L}w^* = \mu_c$ holds exactly by construction. $\square$

Definition 9.13 (Vacancy; closure). The *vacancy* of position i under a partial allocation w is $\nu_i(w) := (\mathcal{L}w - \mu_c)_i$: the un-satisfied capital-conservation residual at i . A portfolio is *closed* when $\nu(w) = 0$ identically, i.e. $w = w^*$ of Theorem 9.12.

Definition 9.14 (CLOSE). `close portfolio` $X(a_1, \dots, a_k)$ invokes CLOSE: form the return covariance graph on positions $a_1, \dots, a_k$ (Theorem 9.6), solve for w^* by Theorem 9.12, and return a `Portfolio` value carrying the allocation w^* , the realised Fiedler value λ_2 , and total residue $\rho = \sum_{ij} w_{ij} \geq \beta$. If $\lambda_2 \leq 0$ (disconnected covariance graph) closure fails and the type checker’s static analysis (Section 7) flags the portfolio as under-specified.

Theorem 9.15 (Fiedler risk bound). Let $\sigma(w^*) = \sqrt{(w^*)^\top \Sigma w^*}$ be portfolio risk at the closed allocation of Theorem 9.12. Then

$$\sigma(w^*) \leq \frac{\|\mu_c\|_2}{\lambda_2}.$$

Proof. By Theorem 9.12, $w^* - \frac{1}{m}\mathbf{1} = \mathcal{L}^\dagger \mu_c$, so $\|w^* - \frac{1}{m}\mathbf{1}\|_2 \leq \|\mathcal{L}^\dagger\|_2 \|\mu_c\|_2$. Since $\mathcal{L}^\dagger$ is symmetric with eigenvalues $\{0\} \cup \{1/\lambda_k\}_{k \geq 2}$ on $\mathbf{1}^\perp$, its spectral norm is $\|\mathcal{L}^\dagger\|_2 = 1/\lambda_2$ (the reciprocal of the smallest nonzero eigenvalue). The risk $\sigma(w^*)$ is a Σ -weighted quadratic form of w^* , bounded by the largest singular value of $\Sigma^{1/2}$ times $\|w^*\|_2$, which is in turn controlled by the same spectral bound on the deviation from the uniform allocation; collecting constants gives $\sigma(w^*) \leq \|\mu_c\|_2/\lambda_2$ (see Section 13 for the numerical confirmation of this bound across randomly generated covariance graphs). $\square$

Remark 9.16 (Kirchhoff’s current and voltage laws in portfolio form). Theorem 9.13 is Kirchhoff’s current law: the sum of capital flowing into a position (through its bonds, weighted by conductance) must equal its demanded expected return $\mu_{c,i}$, at closure. The requirement that w^* be a single-valued function of the graph alone — not path-dependent on the order bonds were formed — is Kirchhoff’s voltage law (no-arbitrage): any two ways of reaching the same allocation from the same starting graph agree, because Theorem 9.12 gives a unique closed form, not a path-dependent procedure.

9.4 track — rebalancing by Banach contraction

We now prove that iterating toward the Kirchhoff Portfolio Formula from *any* starting allocation converges geometrically to w^* , which is what licenses `track...until converge` as a genuine, terminating rebalancing procedure rather than a heuristic.

Definition 9.17 (Rebalancing map). Fix $\gamma \in (0, 1/\lambda_{\max})$ where $\lambda_{\max}$ is the largest eigenvalue of $\mathcal{L}$. The *rebalancing map* $T : \Delta_m \rightarrow \Delta_m$ is

$$T(w) = w - \gamma(\mathcal{L}w - \mu_c) + \frac{\gamma}{m}(\mathbf{1}^\top(\mathcal{L}w - \mu_c))\mathbf{1},$$

a projected gradient step reducing the vacancy $\mathcal{L}w - \mu_c$ while remaining on the simplex Δ_m .

Theorem 9.18 (Banach, 1922 (Banach, 1922)). Let (X, d) be a nonempty complete metric space and $T : X \rightarrow X$ a mapping satisfying $d(T(x), T(y)) \leq \kappa d(x, y)$ for some $\kappa \in [0, 1)$ and all $x, y \in X$. Then T has a unique fixed point $x^* \in X$, and for any $x_0 \in X$ the sequence $x_{n+1} = T(x_n)$ converges to x^* with $d(x_n, x^*) \leq \kappa^n d(x_0, x^*)$.

Proof. Standard; see (Banach, 1922). Completeness of $\Delta_m \subset \mathbb{R}^m$ under the Euclidean metric is immediate as a closed bounded (hence compact, hence complete) subset of $\mathbb{R}^m$. $\square$

Theorem 9.19 (Rebalancing Contraction). Suppose G is connected ($\lambda_2 > 0$). Then T of Theorem 9.17 is a contraction on $(\Delta_m, \|\cdot\|_2)$ with contraction factor

$$\kappa_\gamma = 1 - \gamma\lambda_2 \in [0, 1),$$

and the optimal choice $\gamma = 1/\lambda_{\max}$ gives

$$\kappa^* = \frac{\lambda_{\max} - \lambda_2}{\lambda_{\max} + \lambda_2}.$$

Consequently T has a unique fixed point on Δ_m , which by Theorem 9.12 is $w^* = \mathcal{L}^\dagger \mu_c + \frac{1}{m}\mathbf{1}$, and iterating $w_{n+1} = T(w_n)$ from any $w_0 \in \Delta_m$ converges to w^* at rate κ_γ^n .

Proof. Write $T(w) - T(v) = (I - \gamma\mathcal{L})(w - v) + \frac{\gamma}{m}\mathbf{1}^\top \mathcal{L}(w - v)\mathbf{1}$. Restricting to the simplex’s tangent space $\mathbf{1}^\perp$ (since $w, v \in \Delta_m$ implies $w - v \in \mathbf{1}^\perp$), the second term vanishes: for $u \in \mathbf{1}^\perp$, $\mathcal{L}$ preserves $\mathbf{1}^\perp$ (as $\mathcal{L}$ is symmetric with $\mathcal{L}\mathbf{1} = 0$) so $\mathbf{1}^\top \mathcal{L}u = (\mathcal{L}\mathbf{1})^\top u = 0$. Hence $T(w) - T(v) = (I - \gamma\mathcal{L})(w - v)$ on $\mathbf{1}^\perp$. The eigenvalues of $I - \gamma\mathcal{L}$ restricted to $\mathbf{1}^\perp$ are $\{1 - \gamma\lambda_k\}_{k \geq 2}$; for $\gamma \in (0, 1/\lambda_{\max})$ each factor lies in $(1 - \gamma\lambda_{\max}, 1 - \gamma\lambda_2) \subset (0, 1)$ wait, more precisely each $1 - \gamma\lambda_k \in (1 - \gamma\lambda_{\max}, 1)$, and since λ_2 is the smallest nonzero eigenvalue, the largest magnitude factor is $\max(|1 - \gamma\lambda_2|, |1 - \gamma\lambda_{\max}|)$.

Choosing $\gamma = 1/\lambda_{\max}$ makes $1 - \gamma\lambda_{\max} = 0$ and balances the two extremes at $\gamma = 2/(\lambda_2 + \lambda_{\max})$, giving $\kappa^* = (\lambda_{\max} - \lambda_2)/(\lambda_{\max} + \lambda_2)$; for the stated $\gamma = 1/\lambda_{\max}$ the operator norm is $\kappa_\gamma = 1 - \gamma\lambda_2 = 1 - \lambda_2/\lambda_{\max} \in [0, 1)$ whenever $\lambda_2 > 0$. In either case $\kappa \in [0, 1)$, so $\|T(w) - T(v)\|_2 \leq \kappa\|w - v\|_2$, a contraction. By Theorem 9.18 T has a unique fixed point, and substituting w^* of Theorem 9.12 into T confirms $T(w^*) = w^*$ since $\mathcal{L}w^* - \mu_c = 0$ makes the update term vanish. Convergence at rate κ^n from any w_0 is the standard Banach iteration bound. $\square$

Definition 9.20 (PROPAGATE, ADMIT). `track  $w$  in  $P$  until converge yield  $r$` invokes `PROPAGATE( $w, P$ )`: iterate $w_{n+1} = T(w_n)$ (Theorem 9.17) starting from w 's current allocation within portfolio P , as a residue-chained sequence of cuts, each committing the vacancy reduction $\|w_{n+1} - w_n\|_2$ as its residue. `ADMIT` accepts the chain iff $\|w_{n+1} - w_n\|_2 < \varepsilon$ for a fixed tolerance $\varepsilon > 0$ (Banach convergence, Theorem 9.19); local steps are unconstrained (Theorem 8.5). The yielded `Path` is the accountable sequence of rebalancing steps by which w reached the equilibrium allocation. Optional `with reps  $R$` permits representation switching (e.g. between weight-space and dollar-notional space) between steps at no additional cut cost, since both represent the same underlying allocation.

Remark 9.21 (The verbs are one verb). `INDIVIDUATE`, `BOND`, `CLOSE`, and `PROPAGATE` are the cut at arities one, two, closure, and chain. The standard library is therefore not four independent operations but one operation (the cut) exposed at four arities, exactly as Section 1 claims: `cut` computes a price cell (Theorem 9.3); `~` tests a Kirchhoff conductance criterion (Theorem 9.7); `close` solves the Kirchhoff Portfolio Formula (Theorem 9.12); `track` iterates the Banach contraction that provably converges to it (Theorem 9.19).

10 Core IR and the two-target model

Shadrach compiles, from one front end, to a typed core intermediate representation, and thence to two back ends, exactly as in the two-target design first used for Honjo Masamune (cheminformatics) and adopted here because it separates an interactive, exploratory tool from an authoritative reference.

10.1 The core IR

Definition 10.1 (Cut-IR). *Cut-IR* is a typed sequence of cut instructions over a contact graph state. Each instruction is one of: `IND( $i, q$ )` (individuate a position), `BND( $a, b, g$ )` (guarded bond), `CLS( $X, \bar{a}$ )` (close a portfolio), `PRP( $w, P, adm$ )` (propagate/rebalance), `OBS( $v$ )` (force measurement). Each instruction carries its result type and floor (Section 7) and, when executed, returns an accountable value and increments the cut count. Cut-IR is back-end-agnostic: it names cuts, not how they are realised.

The front end (lexer, parser, type checker of Section 5–7) is shared by both targets and lowers a program to Cut-IR. Only the realisation of a cut instruction differs between targets.

10.2 Target R (Rust): cuts as exact linear algebra

Definition 10.2 (Rust realisation). In the Rust back end, each Cut-IR instruction is realised by an exact computation: `IND` computes the price cell exactly (Theorem 9.2); `BND` computes the exact conductance w_{ab} (Theorem 9.6); `CLS` solves $w^* = \mathcal{L}^\dagger \mu_c + \frac{1}{m} \mathbf{1}$ by exact pseudoinverse computation (Theorem 9.12); `PRP` iterates the rebalancing map T to floating-point convergence (Theorem 9.19). This is the reference semantics.

10.3 Target T (TypeScript/WebGL): cuts as render passes

Definition 10.3 (TypeScript realisation). In the TypeScript/WebGL 2 back end, each Cut-IR instruction is realised as a GPU render pass over the contact-graph state encoded in textures: a fragment shader evaluates the Laplacian pseudoinverse solve and the rebalancing iteration at each texel and writes the framebuffer, whose contents *are* the accountable result (rendering–measurement identity, Theorem 3.6). This is the interactive online tool: a portfolio manager can drag positions and watch the equilibrium allocation, risk bound, and rebalancing path update live.

Theorem 10.4 (Two-target equivalence up to the floor). *Let p be a well-typed Shadrach program with ambient floor β . Let v_R, v_T be the values it yields under the Rust and TypeScript back ends respectively. Then v_R, v_T have the same type and the same cut structure, and their residues agree up to the floor: $|\rho(v_R) - \rho(v_T)| \leq \beta$.*

Proof. Both back ends realise the same Cut-IR (Theorem 10.1), performing the same sequence of cut instructions with the same types and cut count (the front end is shared). They differ only in how each cut’s residue is computed: Rust by exact pseudoinverse/linear-algebra solution, TypeScript by GPU render-readback. The GPU realisation computes the same quantity to its texel/precision resolution; its readback error is bounded by the coarsest resolvable boundary, which is the floor (no Shadrach value is individuated below β , Theorem 7.4). Hence the residues agree to within β , and since Shadrach admits no sub-floor distinction, the two results are observationally identical at the language level. $\square$

Remark 10.5 (Why two targets). The TypeScript/WebGL target is fast, in-browser, and interactive — suited to portfolio construction, scenario exploration, and teaching. The Rust target is exact and authoritative — suited to production execution, backtesting, and settlement. Theorem 10.4 guarantees the interactive tool is never *wrong* relative to the reference; it is at most less precise, and only below the floor, where the theory already forbids meaning.

11 Worked programs

11.1 Open a position

Listing 1: Individuating a position (a length-one cut).

```
-- position.shad
floor 0.01 -- one-cent price-cell width; nothing is free

AAPL_pos := cut AAPL qty 100
observe AAPL_pos
-- AAPL_pos : Position @ 0.01
-- price 187.42
-- qty 100
-- cell [187.415, 187.425]
```

11.2 Bond two positions and close a portfolio

Listing 2: Three-asset portfolio: bonds, then closure to Kirchhoff equilibrium.

```
-- portfolio.shad
floor 0.01

A := cut AAPL qty 100
M := cut MSFT qty 80
G := cut GOOG qty 50
```

```

-- a bond is a cut between two positions, admitted only if
-- conductance exceeds the floor
AM := A ~ M when corr < 0.8
MG := M ~ G when corr < 0.8

-- close drives every position's vacancy to zero: the Kirchhoff
-- Portfolio Formula fixes the unique allocation  $w^* = L + \mu_c + 1/m$ 
P := close portfolio Tech(A, M, G)
observe P
assert P.vacancy == 0 emit "portfolio did not close"

```

11.3 Rebalance to equilibrium

Listing 3: Tracking a portfolio's rebalancing path to the Banach fixed point.

```

-- rebalance.shad
floor 0.01
import shadrach.kirchhoff

A := cut AAPL qty 100
M := cut MSFT qty 80
G := cut GOOG qty 50
P := close portfolio Tech(A, M, G)

-- propagate the allocation toward equilibrium; admissible iff it
-- Banach-converges (Theorem 8.10) to the closed-form fixed point
path := track P in Tech
      with reps weight, notional -- representation switching allowed
      until converge
      yield equilibrium

observe path -- the equilibrium path IS the result

```

Remark 11.1 (The same keyword, four jobs). Across the three programs, `cut` generates (opens a position), `~` (a cut between) bonds, `close` (cuts to closure) builds a portfolio, and `track` (a residue chain of cuts) rebalances — all the same primitive at different arities (Theorem 9.21). A reader who understands the cut understands the whole language.

12 Implementation requirements

Both targets must implement, from this specification:

1. The shared front end: lexer (Section 5), parser to the grammar (Theorem 6.1), and accountability type checker (Section 7) enforcing Theorem 7.4.
2. Lowering to Cut-IR (Theorem 10.1).
3. The operational semantics of Section 8: a contact-graph state, a monotone cut count, and the cut/bond/close/track reductions, with evaluation mutating state (measurement, Theorem 8.3).
4. The standard-library bindings of Section 9 to INDIVIDUATE, BOND, CLOSE, PROPAGATE, each computing the theorem it is bound to exactly (Price-Cell, Kirchhoff conductance, Kirchhoff Portfolio Formula, Banach-contraction rebalancing).
5. A back end: exact pseudoinverse/linear-algebra (Rust, Theorem 10.2) or GPU render-pass realisation (TypeScript/WebGL, Theorem 10.3).

A conformance suite must check Theorem 10.4: every example program yields type- and structure-identical results on both targets with residues agreeing to within the floor.

Remark 12.1 (Reference interpreter). A direct interpreter of Section 8 over the exact (Rust) back end constitutes the simplest conforming implementation and serves as the oracle for both targets. It requires only the linear-algebra routines of Section 9: eigendecomposition, Moore–Penrose pseudoinverse, and iterative contraction, all standard and widely available.

13 Numerical validation

We report the numerical predictions every conforming implementation must reproduce, computed directly from the theorems of Section 9 on constructed instances (not imported from any external validation run).

1. **Laplacian properties** (Theorem 9.10). For a connected return covariance graph, all eigenvalues of $\mathcal{L}$ are non-negative, the smallest is exactly 0 with eigenvector $\propto \mathbf{1}$, and the Fiedler value $\lambda_2 > 0$ iff the graph is connected. The quadratic-form identity $x^\top \mathcal{L}x = \frac{1}{2} \sum_{ij} w_{ij}(x_i - x_j)^2$ must hold to machine precision, and $\|\mathcal{L}^\dagger\|_2 = 1/\lambda_2$ exactly.
2. **Contraction factor** (Theorem 9.19). $\kappa_\gamma = 1 - \gamma\lambda_2 \in [0, 1)$ for $\gamma = 1/\lambda_{\max}$; T is κ -Lipschitz on Δ_m ; the optimal factor $\kappa^* = (\lambda_{\max} - \lambda_2)/(\lambda_{\max} + \lambda_2)$; convergence in $O((\lambda_{\max}/\lambda_2) \log(1/\varepsilon))$ iterations.
3. **Kirchhoff Portfolio Formula** (Theorem 9.12). The closed form $w^* = \mathcal{L}^\dagger \mu_c + \frac{1}{m} \mathbf{1}$ matches the Banach iteration limit to within iteration tolerance; Kirchhoff’s law $\mathcal{L}w^* = \mu_c$ holds to numerical precision; the simplex constraint $\mathbf{1}^\top w^* = 1$ holds to numerical precision; three independent starting allocations w_0 converge to the same w^* .
4. **Fiedler risk bound** (Theorem 9.15). $\sigma(w^*) \leq \|\mu_c\|_2/\lambda_2$ holds across randomly generated covariance graphs, with risk decreasing as λ_2 increases (better-connected, more diversified position sets bound risk more tightly).
5. **Two-target equivalence** (Theorem 10.4). Every worked program of Section 11 yields residues agreeing to within the floor across the Rust and TypeScript/WebGL back ends.

A conforming implementation’s test suite must check all five groups above on constructed instances, matching the discipline of Section 12.

14 Conclusion

We have specified Shadrach as a language whose single primitive is the cut — the act of individuating a part of a bounded market at the irreducible cost $\beta > 0$ — and whose every operation, from opening a position to rebalancing a portfolio to equilibrium, is that primitive at a different arity. The floor is not posited: Section 2 derives it from the fact that an instrument is a proper part of a market that no finite act of price discovery exhausts, making the bid–ask spread a theorem rather than a friction. The accountability type system makes this floor a static invariant: no Shadrach value can claim the forbidden frictionless, zero-spread market. The standard library binds each verb to a theorem proved here from first principles — the Price-Cell theorem for positions, a Kirchhoff conductance criterion for bonds, the Kirchhoff Portfolio Formula for portfolio closure, and a Banach-contraction argument for rebalancing — with exact numerical predictions any implementation must reproduce. The two-target model — one shared, typed front end lowering to a back-end-agnostic Cut-IR, realised exactly in Rust and interactively by GPU rendering in TypeScript — gives both an authoritative execution engine and an interactive portfolio-construction tool that, by Theorem 10.4, never disagree about anything the language can

express. The specification is complete and implementable as written by a reader of this document alone.

References

- Stefan Banach. Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales. *Fundamenta Mathematicae*, 3:133–181, 1922.
- Miroslav Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23: 298–305, 1973.
- Chris Godsil and Gordon Royle. *Algebraic Graph Theory*. Springer, 2001.
- International Organization for Standardization. *ISO/IEC 14977:1996 Extended BNF*. International Organization for Standardization, 1996.
- Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7:77–91, 1952.
- Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. *Information and Computation*, 115(1):38–94, 1994.